

PRODUCT REQUIREMENTS DOCUMENT

OneStop News

A topic-first news aggregation and AI summarization platform that organizes content by what it is about, delivering source-cited, disclosure-first intelligence in an editorial interface.

VERSION

2.1 — Research-Validated Refinement

DATE

June 2026

STATUS

Production-Aligned

Table of Contents

1. Overview	4
1.1 Architectural Commitment (Updated and Definitive)	4
1.2 Scope	5
2. Goals and Success Metrics	5
2.1 Product Goals	5
2.2 Scale Assumptions	6
2.3 Success Metrics (V1)	6
3. Target Users and Personas	7
3.1 Daily Scanner	7
3.2 Enterprise Analyst / Researcher	7
3.3 Editor / Admin	7
4. Use Cases and User Stories	7
4.1 Topic-First Browsing	7
4.2 Search and Sorting	8
4.3 Article Exploration and Detail	8
4.4 AI Summarization - Source-Cited Model	8
4.5 Source Transparency	8
4.6 Admin and Operations	9
5. Information Architecture and Navigation	9
5.1 Topic Model	9
5.2 Navigation Model	9
5.3 URL and Routing Schema	9
6. UX and UI Requirements	10
6.1 Design Philosophy - The Conceptual Direction	10
Aesthetic Directives	10
Typographic System	10
Color System (Design Token Names)	10

Dark Mode	11
6.2 Layout System	11
Desktop ($\geq 1280\text{px}$)	11
Tablet (768px - 1279px)	11
Mobile ($< 768\text{px}$)	12
6.3 AI Summary UX - Source-Cited Disclosure Model	12
6.4 Accessibility	12
7. Functional Requirements	13
7.1 Ingestion Pipeline	13
Ingestion Job Steps	13
7.2 Article Lifecycle	13
7.3 Ranking and Importance Score	13
7.4 Summarization - Source-Cited Model	14
Summarization Job Steps	14
7.5 Search and Filtering	14
7.6 User Features	15
V1	15
Roadmap	15
7.7 Admin Features	15
8. System Architecture	16
8.1 High-Level Architecture	16
8.2 Internal Layering	16
8.3 Next.js 16 Caching Strategy	17
8.4 Data Flow	17
Ingestion Flow	17
Feed Query Flow	18
Summarization Flow	18
9. Data Model and Storage	18
9.1 Core Schema (Drizzle ORM)	18

9.2 Indexing Strategy	19
9.3 No SQLite in Production	20
10. API Design	20
10.1 HTTP Endpoints (Route Handlers)	20
Internal Mutations (Server Actions, not Route Handlers)	20
10.2 Authentication and Authorization	20
10.3 Error Handling	20
11. Caching, Performance and Scalability	21
11.1 Caching Strategy	21
11.2 Performance Targets	21
11.3 Scalability	21
12. Observability and Operations	22
12.1 BullMQ Dashboard	22
12.2 Metrics	22
12.3 Logging and Tracing	22
12.4 Alerting	22
13. AI Governance	23
13.1 Disclosure Policy (Research-Grounded)	23
13.2 Summary Quality Standards	23
13.3 Quality Control Process	23
13.4 Security	23
14. Rollout Plan	24
Phase 1 - Production Foundation (MVP)	24
Phase 2 - Performance and Personalization	24
Phase 3 - Semantic Intelligence and Enterprise	24
15. Risk Register	24
Appendix A: Technology Stack Summary	25
Appendix B: Key Research Findings	26

1. Overview

OneStopNews is a **topic-first news aggregation and AI summarization platform** that organizes public news content by what it is about rather than who published it. It collects article metadata from many sources, normalizes and categorizes stories, and presents them in a calm, editorially-informed interface designed for both daily readers and enterprise analysts. The platform distinguishes itself in a crowded market by combining AI-powered topic clustering and summarization with a source-cited, transparency-first disclosure model that builds trust rather than eroding it.

The competitive landscape in 2025-2026 reveals a significant market gap: no existing platform combines AI summarization, bias detection, topic clustering, and source-first control in a single product at consumer-accessible price points. While platforms like Particle and Kagi News offer AI-first news reading, and Ground News provides media bias analysis, none integrate these capabilities holistically. Enterprise-grade tools like Bloomberg Terminal offer sophisticated AI summaries and entity-centric organization but at prohibitive costs (\$24,000/year). OneStopNews targets this white space by delivering enterprise-caliber intelligence in an editorial interface accessible to everyday readers and professional analysts alike.

The global news aggregator market is estimated at USD \$13.5 billion in 2025, projected to reach USD \$32 billion by 2033 (CAGR 11.2%). Monetization is shifting toward subscriptions: only 18% of people paid for online news in 2025 (Reuters Institute), making conversion the critical bottleneck. The freemium-plus-AI-upsell model pioneered by Feedly represents the most proven path, and OneStopNews follows this pattern with a free topic-first reading experience and premium AI-powered features.

1.1 Architectural Commitment (Updated and Definitive)

This document makes the following **definitive architectural choices**, replacing the speculative language of the original draft. Each choice is grounded in production research conducted across six technical dimensions in June 2026.

Concern	Choice	Rationale
Web framework	Next.js 16 (App Router, PPR, Cache Components)	Production-stable; new opt-in caching model aligns with mixed static/dynamic feeds
UI runtime	React 19.2 (View Transitions, Activity)	View Transitions for feed navigation; Activity for background summary loading
Styling	Tailwind CSS v4 + custom design tokens	Utility-first with zero generic defaults; v4 engine is faster and more composable
Components	Shadcn UI (Radix primitives)	Library-first approach; wrapped for bespoke "Editorial Dispatch" aesthetic
Job queue	BullMQ on Redis	14M+ npm downloads/month; purpose-fit for complex job graphs, priorities, and built-in dashboard
Database	PostgreSQL 17 (primary)	Only production-supported datastore; no schema drift from SQLite

FTS	GIN-indexed tsvector + pg_textsearch BM25	Elasticsearch-free; pg_textsearch v1.0 production-ready (138M docs benchmarked)
ORM	Drizzle ORM	TypeScript-native, strict-mode compatible, lightweight
Auth	Auth.js v5 (session cookies, HttpOnly)	Modern, Next.js 16-native, supports SAML/OIDC roadmap
Worker runtime	Node.js 24+	LTS-aligned; BullMQ-native; optimal for I/O-bound ingestion and AI-API-bound summarization
Validation	Zod	Schema-first, composable, Drizzle-compatible
Network boundary	proxy.ts (not middleware.ts)	Next.js 16 replaces middleware with proxy for clarity at the network boundary

Table 1: Definitive Architecture Choices

Removed from consideration: SQS (fire-and-forget only, lacks job priorities and native dashboard), RabbitMQ (operational complexity exceeds team size), SQLite in production (out of scope for schema consistency).

1.2 Scope

This PRD covers product requirements (features, UX, roles) for OneStopNews, a concrete production-grade architecture, and non-functional requirements for performance, reliability, observability, and AI governance. The document also incorporates competitive analysis insights from the 2025-2026 news aggregator market, particularly the identification that no existing platform combines AI summarization with bias detection, topic clustering, and source-first control.

Out of scope (this iteration): Full microservice decomposition, dedicated data warehouse, advanced ML-based personalization algorithms, pgvector semantic search (roadmap Phase 3), and native mobile applications (the responsive web application is the primary deliverable).

2. Goals and Success Metrics

2.1 Product Goals

- Provide a **topic-first news reading experience** that reduces cognitive load and tab-hopping across publisher sites, addressing the "overwhelming volume" complaint identified as the top user frustration in 2025 research.
- Offer **source-cited, disclosure-first AI summaries** that speed up comprehension while preserving the primacy of original articles. Research from the University of Florida (2026) confirms a "collaboration premium" where AI+human content is trusted more than either alone, and that source citations counteract the "disclosure penalty" that normally erodes trust.
- Achieve enterprise-grade reliability and observability across ingestion, search, and summarization pipelines, with BullMQ providing real-time visibility into job states.
- Maintain a **distinct editorial-typographic visual identity** that rejects the homogenized "AI slop" aesthetic and instead follows the "Editorial Dispatch" conceptual direction with intentional minimalism

and bespoke typography.

- Establish a **bias-awareness layer** as a roadmap feature, inspired by Ground News which is the only platform currently providing systematic bias ratings and blindspot detection. This is a differentiator in the market.

2.2 Scale Assumptions

Dimension	Target
Sources	50-200 RSS/Atom/API sources
Ingestion volume	20k-100k candidate articles/day post-dedup
Peak burst	3-5x normal volume during major news events
MAU (early enterprise)	Up to low-hundreds of thousands
Read/write ratio	~95% reads - justifies aggressive feed caching

Table 2: Scale Assumptions

2.3 Success Metrics (V1)

Metric	Target	Measurement
Feed freshness	95% of category feeds show ≥ 20 stories from last 24h	Worker job monitoring
API p95 latency	≤ 500 ms server time for GET /api/articles under normal load	APM tracing
Page render p95	≤ 1.5 s for main feed (PPR + Cache Components)	Core Web Vitals / RUM
Summarization coverage	30-50% of viewed high-interest articles have summaries within 24h	DB analytics
Summary trust rate	$< 1\%$ of audited summaries flagged for material factual errors	Manual sampling
Availability	99.5% monthly for read APIs + ingestion	Uptime monitoring
AI trust disclosure compliance	100% of summaries shown with source citations visible	UI audit
Citation presence trust uplift	Measure user trust delta: summary with citations vs. without	A/B testing (Phase 2)

Table 3: Success Metrics

The last two rows are research-grounded additions: citations counteract the "disclosure penalty" (Pew Research found only 1% of users click citations, but their presence functions as a trust signal), and measuring the trust uplift validates the core AI disclosure strategy.

3. Target Users and Personas

3.1 Daily Scanner

The Daily Scanner checks news multiple times daily, skims headlines and excerpts, and occasionally opens original articles. This persona thinks in topics ("What is happening with AI regulation?") rather than outlets, which is the dominant behavior pattern identified in 2025 competitive research where topic-first organization is winning for casual users over source-first RSS models. The Daily Scanner needs a fast, clean interface on both mobile and desktop, and appreciates AI summaries as time-savers when clearly sourced and disclosed. This persona is particularly sensitive to the "duplication fatigue" complaint where the same story is covered 20+ times across sources, and values deduplication and multi-source clustering above all.

3.2 Enterprise Analyst / Researcher

The Enterprise Analyst works in finance, policy, or corporate strategy and monitors specific companies, sectors, and regions continuously. This persona needs trustworthy topic grouping, accurate timestamps, source attribution, and AI summaries with citations to specific sources used, not a black-box compression. Research into enterprise platforms reveals that Bloomberg Terminal provides "AI Summary for Company News" and entity-centric organization at \$24,000/year, but these capabilities are not available at consumer price points. The Enterprise Analyst represents the premium tier: they will pay for structured summarization, cross-document synthesis, and temporal analysis of how stories develop over time. Key requirements include export capabilities, advanced filtering, and the ability to trace every summary claim back to its source article.

3.3 Editor / Admin

The Editor manages sources, categories, and ingestion policies, monitors system health via BullMQ dashboard and application metrics, and reviews AI summaries for quality. This persona is responsible for flagging, disabling, or regenerating problematic summaries. The admin also manages the AI governance pipeline: ensuring that every summary meets the disclosure requirements mandated by the EU AI Act Article 50 (effective August 2026) and California SB 942 (effective October 2025). The admin needs a clear view of summarization audit data including volume, error rates, cost proxies (token usage by model), and per-category breakdowns. BullMQ Board provides real-time visibility into queue depths, stuck jobs, and failure rates, which is critical for a 200-source ingestion system with summarization pipelines.

4. Use Cases and User Stories

4.1 Topic-First Browsing

- As a user, I can select a topic (e.g., Tech News) and a subtopic (e.g., AI and ML) to see the latest stories across sources.
- As a user, I can quickly switch topics via a sticky topic navigation ribbon with live story counts.
- As a user, I can see a "Full Coverage" view for any story that groups all source articles covering the same event, inspired by Google News's multi-perspective presentation.

4.2 Search and Sorting

- As a user, I can search across all stories by keyword with BM25-ranked relevance results.
- As a user, I can filter results by category, subcategory, and time range.
- As a user, I can sort results by: Latest, Impact (importance score), or Summary Ready.

4.3 Article Exploration and Detail

- As a user, I can see a lead story for a topic and a dense grid of cards for remaining stories.
- As a user, I can click a card to see a detail view with metadata, an AI summary (if available) with source citations, and a link to the original article.

4.4 AI Summarization - Source-Cited Model

This section is significantly refined from the original draft based on AI trust research. The key insight from the University of Florida (2026) is the "collaboration premium": AI+human content is trusted more than either alone. However, disclosing AI involvement often reduces trust even when content is identical (the "disclosure penalty"). The solution is pairing mandatory disclosure with trust-building features: source citations, human oversight badges, and confidence indicators. Pew Research found that only 1% of users click citation links, but their presence functions as a trust signal far more than their use.

- As a user, I can request an AI summary for an article without one.
- As a user, every AI summary I see shows: which sources it was derived from, a confidence indicator, and a "Read original" escape hatch, never just a floating claim.
- As a user, I can toggle between "AI Summary" and "Original Excerpt" views.
- As an admin, I can review sampled summaries, mark problematic ones, and trigger regeneration.
- As an admin, I can see token usage, model used, and generation timestamp for every summary.
- As a user, I can see an "Opposite Sides" perspective view for polarized topics, inspired by Particle News which groups multi-source coverage and shows contrasting perspectives.

4.5 Source Transparency

- As a user, I can see the source outlet, category, subcategory, and time-ago for each article.
- As a user, I can click "Open original source" to read the full article on the publisher site.
- As a user, I can see source ownership data and media bias indicators for configured sources (Phase 2 feature inspired by Ground News).

4.6 Admin and Operations

- As an admin, I can configure sources (URLs, polling intervals, default categories).
- As an admin, I can view a BullMQ dashboard showing ingestion and summarization queue states: stuck jobs, failed jobs, and throughput.
- As an admin, I can view application metrics (ingestion lag, summary error rates, API latency).

5. Information Architecture and Navigation

5.1 Topic Model

Categories and subcategories are stored in the database rather than hard-coded, to support evolution as new topics emerge and existing ones merge. The initial taxonomy reflects the Singapore-centric market focus while accommodating global coverage.

Category	Example Subcategories
Top Stories	All, Breaking, Editor's Picks
Local News	Transport, Housing, Local Business, Governance
Tech News	AI and ML, Apple and Devices, Startups, Cybersecurity, Open Source
Global News	China, US, Asia-Pacific, Europe, Middle East
Finance News	Markets, Earnings, Personal Finance, Crypto, Commodities
Politics	SG Politics, US Politics, China Politics, Geopolitics
Culture and Society	K-Culture, Internet Culture, SG Gossip, Global Entertainment

Table 4: Topic Taxonomy

5.2 Navigation Model

- Sticky topic navigation ribbon near top of the app workspace, following the three-column adaptive pattern (Sidebar - List - Reading pane) validated by Apple HIG Split Views and the Reeder app.
- Each topic opens a panel with description and subtopic grid with per-subtopic story counts.
- Controls panel shows current view, selected category, result count, filters, and sort options.
- Dual-pane on large screens (feed + sticky detail panel); stacked on smaller screens with bottom-sheet for article detail on mobile.
- View Transitions API (React 19.2) used for topic-switching animations: purposeful, not decorative, respecting prefers-reduced-motion.

5.3 URL and Routing Schema

```
/ -> Default feed (Top Stories / All)
```

```
/topics/[category] -> Default subtopic for category
/topics/[category]/[sub] -> Feed filtered by both
/article/[id] -> Standalone article detail (deep link, shareable)
/admin -> Admin workspace (protected)
/admin/sources -> Source management
/admin/jobs -> Ingestion + summarization job monitor
```

6. UX and UI Requirements

6.1 Design Philosophy - The Conceptual Direction

"Editorial Dispatch - The aesthetic of a wire service terminal meets a refined long-read publication. Not a news app. A briefing room. Every element has the weight of something worth reading."

Aesthetic Directives

- **NOT:** Rounded cards, purple gradients, Inter font at 16px, hero sections with stock photography, predictable 3-column grids. These are the hallmarks of "AI slop" that 2025-2026 design discourse explicitly rejects.
- **YES:** Tight typographic hierarchy, ink-weight contrast, structured information density, purposeful whitespace as a grid element, category-coded color accents that feel like editorial ink rather than UI chrome.

Typographic System

Typography is taking center stage in 2026, moving "beyond legibility into storytelling" (Figma 2026 Design Trends). The anti-generic mandate requires rejecting Inter, Roboto, and system-ui as primary typefaces in favor of deliberate, personality-driven pairings.

Role	Typeface	Rationale
Headlines / Lead stories	Newsreader (editorial serif)	Tight tracking, high contrast stroke weight; echoes classic newspaper broadsheet styling
UI / Body / Labels	Space Grotesk (non-generic grotesk)	Distinct letterforms; not the safe Inter/Roboto default; pairs well with editorial serif
Monospace accents	JetBrains Mono	Timestamps, source tags, metadata; adds wire-terminal feel

Table 5: Typographic System

Color System (Design Token Names)

Research from Google's Material 3 Expressive study found that users of all ages prefer design that makes them feel something: using color, size, shape, and containment to help users navigate and find key elements. The Dispatch color system applies this: each category has a distinctive but restrained accent that

communicates without overwhelming. Semantic color tokens follow the tier system (primitive tokens, semantic tokens, component tokens) for seamless light/dark mode transitions.

Token	Purpose	Value Concept
--ink-900	Near-black text (headlines)	Darkest readable text
--ink-600	Body text	Comfortable reading weight
--ink-300	Muted / metadata	Recessive information
--paper-50	Background	Warm off-white, not clinical white
--paper-100	Card surface	Subtle elevation over background
--dispatch-amber	Breaking / high-impact accent	Urgency without alarm
--dispatch-sage	Finance / stable category	Calm, established
--dispatch-slate	Tech / neutral category	Intelligence, precision
--dispatch-clay	Politics / warm category	Gravity, institutional
--dispatch-violet	Culture / expressive category	Creativity, range

Table 6: Color Token System

Dark Mode

Dark mode is no longer a novelty in 2025; it is an expectation. The system must be designed from day one, not bolted on. Following editorial dark mode best practices: the ideal dark surface color is #121212 (not pure black #000000) to reduce halation around text. Brand colors need systematic adjustment for dark backgrounds: same hues, different brightness/shade, never simple inversion. The dark mode uses elevated surfaces at #1E1E1E and #2C2C2C with off-white text (#E0E0E0), and accent colors saturated for visibility on dark backgrounds.

6.2 Layout System

Desktop (>= 1280px)

- 12-column grid, max-width: 1440px.
- Left: Topic nav ribbon (fixed, 240px) + feed (8 cols). Right: Sticky detail panel (4 cols) for article detail, summary, and source info.
- Lead story: full-width editorial block with large serif headline and image. Sub-stories: 3-column dense grid of compact cards (Bento grid pattern).
- This three-column adaptive pattern (Sidebar, List, Reading pane) follows the master-detail pattern validated by Apple HIG Split Views and the Reeder app gold standard.

Tablet (768px - 1279px)

- Collapsible topic nav (full-width drawer on demand). Single-column feed; detail view as overlay or separate route.

Mobile (< 768px)

- Stacked: sticky mini-header, topic carousel, controls strip, feed, detail. Progressive disclosure and thumb-friendly bottom navigation. Bottom-sheet for article detail.
- Minimum 48px touch targets (validated by Google M3 Expressive research as bridging the age gap in UI interaction speed).

6.3 AI Summary UX - Source-Cited Disclosure Model

This section is the most significantly updated area of the PRD, grounded in the "disclosure penalty" research. The central paradox: regulations mandate disclosure (EU AI Act Article 50, California SB 942), but disclosure reduces trust. The solution is pairing mandatory disclosure with trust-building features that offset the penalty.

Every AI summary component **MUST** include:

- **"AI-generated summary" label** - prominent, not buried. Uses sparkle icon + text, following Google AI Overviews pattern.
- **Source citations inline** - numbered superscripts linking to specific articles used, following Perplexity's hover-preview gold standard. Shape of AI library identifies 4 citation variations: inline highlights, direct quotations, multi-source references, and lightweight links.
- **Model + generation date** - surfaced in a collapsible "About this summary" panel.
- **"Read original source"** - always a primary action, never hidden. This is the "escape hatch" that preserves user agency.
- **Confidence/coverage indicator** - shows how much of the original content was available to the model. Six pattern types identified: progress bars, color coding, opacity gradients, verbal labels, tooltip explanations, and confidence-based escalation.
- **"Editor Reviewed" badge** - when applicable, signals human oversight which creates the "collaboration premium" trust effect.

6.4 Accessibility

- WCAG AA minimum for all contrast ratios; target AAA for body text (4.5:1 minimum, 7:1 target).
- All interactive elements keyboard-accessible with visible focus indicators.
- ARIA roles for topic nav (navigation), article cards (article), summary panels (region with aria-label="AI-generated summary").
- prefers-reduced-motion respected: View Transitions disabled when set, no auto-playing video or animations in the feed.
- Dynamic text scaling that reflows layout (not just zoom), following 2025 accessibility best practices.
- Semantic HTML for screen reader navigation of article hierarchies.

7. Functional Requirements

7.1 Ingestion Pipeline

The ingestion pipeline collects article metadata from configured sources via RSS, Atom, JSON APIs, or custom adapters. BullMQ scheduler launches ingestion jobs per-source on configurable schedules (5-30 minute intervals by priority tier). The architecture follows the tiered approach validated for 100+ source systems: centralized scheduler maintains publisher registry with adaptive polling intervals, Feed Fetcher workers pull from queue, and ETag/Last-Modified headers skip unchanged feeds to minimize unnecessary network traffic.

Ingestion Job Steps

- Load source config from database.
- Fetch feed/API with timeout (10s) and exponential backoff retry (3 attempts).
- Parse and normalize to unified ArticleCandidate schema (Zod-validated).
- Deduplicate via canonical URL normalization + content hash.
- Apply category/subcategory mapping via source config + rule engine.
- Persist via Drizzle ORM to PostgreSQL.
- Update SourceHealthSnapshot (last success, failure count, lag).
- Enqueue compute-importance job for new articles.
- All job failures surface as dead-letter queue entries with full error context.

7.2 Article Lifecycle

- Status flow: pending -> active -> archived.
- Deduplication group: near-duplicates from different sources clustered by canonical_url hash and title similarity.
- content_availability flag: title_only | excerpt | partial_text | full_text.
- Title, excerpt, and metadata can be patched if source feed changes.

7.3 Ranking and Importance Score

The importance_score is a composite float (0.0-1.0) that determines story ordering within feeds. It combines four weighted factors designed to surface the most relevant and timely content while preventing stale stories from dominating.

Factor	Weight	Description
Recency	40%	Exponential decay from published_at; recent stories score higher
Source priority	20%	Configurable per-source tier (Tier 1-3); premium outlets get a boost
Cluster size	25%	How many sources cover this story; multi-source coverage signals importance

Category relevance	15%	Manual boost/penalty by category for editorial curation
--------------------	-----	---

Table 7: Importance Score Components

Scoring runs as a BullMQ compute-importance job, triggered on ingestion and periodically recomputed for hot feeds (top 200 articles per category). Scores are written to articles.importance_score with no separate feed table in V1.

7.4 Summarization - Source-Cited Model

Users request summaries via article detail view. The request is validated and enqueued as a BullMQ summarize-article job on the high-priority queue, returning a jobId for polling. The summarization pipeline is designed for the "fidelity over brevity" principle: the goal is clarity without distortion, where shorter must still mean true.

Summarization Job Steps

- Fetch article + source record from database.
- If content_availability is insufficient: attempt full-text retrieval via safe extractor with robots.txt compliance check.
- Run structured prompt through AI model (Claude 3.5 Haiku or GPT-4o-mini for cost efficiency) with fidelity-over-brevity instruction, source citation extraction instruction, and factual-only (no interpretation) constraint.
- Parse model response into structured output: summary_text, key_points array, sources_cited array, model, tokens_used, generated_at.
- Store in summaries table; update articles.has_summary = true, articles.summary_status = "ok".
- UI refresh: article detail view uses React 19.2 Activity component to render summary panel in background and surface it when ready without layout shift.

Admin can set summary_status to "needs_review" or "disabled". The admin summarization audit trail includes volume, error rates, cost proxies (token usage by model rate), and per-category breakdowns.

7.5 Search and Filtering

The search strategy is updated from the original draft to reflect the current PostgreSQL FTS ecosystem. The key finding: pg_textsearch v1.0 (released March 2026 by Timescale/Tiger Data) provides production-ready BM25 ranking with 2.4x-6.5x faster performance than ParadeDB/Tantivy on 2-4 term queries and 8.7x higher throughput for single-term queries. However, no BM25 extensions are currently available on major managed PostgreSQL providers (Supabase, Neon, AWS RDS), so self-hosted PostgreSQL is required for advanced search.

Layer	Technology	Purpose
Primary FTS	PostgreSQL tsvector generated columns + GIN indexes	Fast keyword matching with title (A) and body (B) field weighting

Relevance ranking	pg_textsearch BM25 extension	Production-ranked search without Elasticsearch; 138M docs benchmarked
Fuzzy / autocomplete	pg_trgm trigram similarity	Zero-result query fallback and search-as-you-type
Semantic (roadmap)	pgvector HNSW index + Reciprocal Rank Fusion	Semantic similarity merged with keyword results via RRF (k=60)

Table 8: Search Architecture Layers

Critical optimization: stored tsvector columns with GIN indexes and fastupdate=off achieve 50x speedup over on-the-fly to_tsvector() computation (10M rows: 41s -> 0.88s). Filters include category, subcategory, time range, summary status, and source tier. Sort options: latest, impact, summary ready. Search is debounced (300ms) client-side; results streamed via React Suspense.

7.6 User Features

V1

- Session-persisted default topic, subtopic, and sort preference.
- Basic preference model (favorite categories stored in user_preferences).
- Density toggle: Compact list / Card grid / Magazine layout (following Feedly's validated pattern).

Roadmap

- Read-later list, muted sources, saved searches.
- Topic alerts (as a separate notification service).
- Semantic search via pgvector (Phase 3).
- Media bias indicators and blindspot detection for configured sources (inspired by Ground News, Phase 2).

7.7 Admin Features

- Source management: CRUD for sources with Zod-validated fields; enable/disable toggle.
- Job monitor: Embedded BullMQ Board view showing queue depths, job states, failure rates, and DLQ entries for ingestion and summarization queues.
- Summarization audit: Volume, error rates, cost proxies (token usage by model rate), per-category breakdown.
- Summary review queue: Surface summary_status = "needs_review" items with regenerate/disable actions.

8. System Architecture

8.1 High-Level Architecture

The system comprises three primary components: the Next.js 16 Web App (handling all client-facing routes, server-side rendering, and API endpoints), PostgreSQL 17 (the sole production datastore), and the Worker Service (Node.js 24+ processes consuming BullMQ queues for ingestion, scoring, and summarization). Redis serves as the BullMQ backend and optional feed slice cache. All inter-service communication flows through BullMQ queues, with Drizzle ORM providing the database access layer.

Component	Technology	Responsibility
Web App	Next.js 16 (App Router, PPR, Cache Components)	SSR, API routes, auth, server actions, proxy.ts
Database	PostgreSQL 17	Articles, summaries, sources, GIN FTS indexes, BM25 extension
Queue Backend	Redis (Upstash / Managed)	BullMQ queues, feed slice cache, optional session store
Worker Service	Node.js 24+ / TypeScript	Ingestion jobs, importance scoring, summarization, health snapshots
AI Client	Anthropic SDK + OpenAI SDK	Structured summarization with Zod-validated output

Table 9: System Components

8.2 Internal Layering

Following the feature-based architecture pattern that dominates production Next.js in 2025-2026, the codebase is organized into feature modules with clear boundaries. Server Components handle data-fetching and layouts, Client Components are used only for interactivity islands, Server Actions replace traditional API routes for mutations and authenticated data fetching, and Route Handlers are reserved for webhooks and public endpoints.

```
src/  
app/ <- Next.js App Router (routes, layouts)  
(public)/ <- Public routes group  
page.tsx <- / Default feed (PPR + Cache Components)  
topics/[category]/  
article/[id]/  
(admin)/ <- Admin routes (auth guard via proxy.ts)  
api/ <- Public Route Handlers only  
features/ <- Feature modules (key pattern)  
feed/components/ <- Feed, ArticleCard, LeadStory  
feed/actions.ts <- Server Actions for feed mutations  
feed/queries.ts <- DB queries via Drizzle
```

```

summaries/components/ <- SummaryPanel, CitationList, DisclosureBadge
search/
admin/
domain/ <- Pure business logic (no framework deps)
lib/ <- Infrastructure integrations
db/ <- Drizzle client + schema
queue/ <- BullMQ client + job type definitions
ai/ <- AI client wrapper
auth/ <- Auth.js v5 config
shared/components/ <- Design system (Shadcn wrapped)
proxy.ts <- Next.js 16 network boundary

```

8.3 Next.js 16 Caching Strategy

The caching strategy is fully updated from the original draft, which was based on the old implicit caching model. Next.js 16 Cache Components make caching entirely opt-in: all dynamic code in any page, layout, or API route executes at request time by default. The new APIs include `cacheLife` profiles, `cacheTag`, `updateTag()` for read-your-writes, `refresh()` for uncached data, and updated `revalidateTag(tag, profile)`. The `cacheComponents: true` config enables PPR as default: components with "use cache" become part of the static shell; components without it become dynamic holes wrapped in `Suspense`.

Route	Strategy	Rationale
/ (Top Stories)	PPR + CacheComponent for feed shell, dynamic for counts	Shell prerendered; story counts dynamic
/topics/[category]	PPR + Cache Component (revalidate: 120s)	Relatively stable; tolerate 2-min lag
/article/[id]	Dynamic (always fresh)	Summary status changes; must be current
GET /api/articles	No Route Handler cache; Redis feed slices for hot categories	Feed freshness SLA
GET /api/categories	Cache Component (revalidate: 300s)	Rarely changes

Table 10: Caching Strategy by Route

Redis feed slices: Hot categories (Top Stories, Tech, Finance) maintain a pre-computed ordered list of article IDs in Redis, refreshed by the worker after each ingestion batch. Feed queries hit Redis first, PostgreSQL as fallback. TTL: 5 minutes. Cache invalidation: worker calls `revalidatePath` after ingestion completes per category.

8.4 Data Flow

Ingestion Flow

```

BullMQ Scheduler (repeatable job)
-> Worker: fetch source -> parse -> normalize -> deduplicate -> write PG
-> Worker: update SourceHealthSnapshot

```

```
-> Enqueue: compute-importance jobs for new articles
-> Enqueue: refresh-feed-slice for affected category
```

Feed Query Flow

```
User request -> Next.js RSC
-> Check Redis feed slice (hot category)
-> Miss: Drizzle ORM -> PG (articles + importance_score + has_summary)
-> Return: streamed with Suspense boundaries
```

Summarization Flow

```
User action -> POST /api/summarize/[id] -> validate -> BullMQ enqueue (high
priority)
-> Worker: fetch article -> extract content -> AI model (Zod-validated response)
-> Store: summaries table with sources_cited[]
-> Update: articles.has_summary = true
-> Redis: invalidate article cache key
-> Client: React 19.2 Activity renders summary when polled/revalidated
```

9. Data Model and Storage

9.1 Core Schema (Drizzle ORM)

The schema is defined using Drizzle ORM with TypeScript-native type safety. Key design decisions: generated tsvector columns for automatic FTS index maintenance without manual triggers, composite indexes aligned with the primary read patterns (category + published_at DESC), and a dedicated summaries table with structured source citation storage.

```
-- sources table
id: uuid (defaultRandom, PK)
name: text (notNull)
url: text (notNull, unique)
feedUrl: text (notNull)
feedType: enum rss | atom | json_api (notNull)
categoryId: uuid -> categories.id
priority: integer (default 2) -- 1=high, 2=normal, 3=low
pollIntervalMinutes: integer (default 15)
isActive: boolean (default true)
```

```
-- articles table
id: uuid (defaultRandom, PK)
sourceId: uuid -> sources.id (notNull)
categoryId: uuid -> categories.id
subcategoryId: uuid -> subcategories.id
```

```

title: text (notNull)
excerpt: text
canonicalUrl: text (notNull, unique index)
contentHash: text (notNull)
contentAvailability: enum title_only|excerpt|partial_text|full_text
importanceScore: real (default 0.5)
hasSummary: boolean (default false)
summaryStatus: enum none|pending|ok|needs_review|disabled
publishedAt: timestamp (notNull)
ingestedAt: timestamp (defaultNow)
searchVector: tsvector GENERATED ALWAYS AS
setweight(to_tsvector('english', coalesce(title,'')), 'A') ||
setweight(to_tsvector('english', coalesce(excerpt,'')), 'B')

```

```

-- summaries table
id: uuid (defaultRandom, PK)
articleId: uuid -> articles.id (notNull, unique)
summaryText: text (notNull)
keyPoints: jsonb -> string[] (default [])
sourcesCited: jsonb -> {url, title}[] (default [])
model: text (notNull)
tokensUsed: integer
generatedAt: timestamp (defaultNow)
status: enum ok|needs_review|disabled
flagReason: text

```

9.2 Indexing Strategy

Index	Type	Purpose
(category_id, published_at DESC)	Composite B-tree	Primary feed read pattern
(subcategory_id, published_at DESC)	Composite B-tree	Subtopic feed queries
search_vector	GIN (fastupdate=off)	Full-text search; 50x faster than on-the-fly tsvector
canonical_url	Unique B-tree	Deduplication
search_vector WHERE published_at > NOW() - 7d	Partial GIN	Recent-content search optimization

Table 11: Indexing Strategy

Critical optimization: GIN indexes with fastupdate=off are essential for read-heavy news workloads. The Neon benchmark showing 50x speedup (41s to 0.88s on 10M rows) was achieved with stored tsvector columns and fastupdate=off. The partial index on recent content (7-day window) further reduces index size

by 30-50% for the most common search pattern. For monthly-partitioned tables, per-partition GIN indexes provide optimal maintenance and query performance.

9.3 No SQLite in Production

PostgreSQL 17 is the only production datastore. Local development uses a PostgreSQL container via Docker Compose. SQLite is explicitly not supported to avoid schema drift and migration inconsistencies that arise from maintaining two different SQL dialects. This decision is reinforced by the need for generated tsvector columns, GIN indexes, and BM25 extensions that are PostgreSQL-specific features with no SQLite equivalents.

10. API Design

10.1 HTTP Endpoints (Route Handlers)

Method	Endpoint	Description
GET	/api/categories	Categories + subcategories with article counts
GET	/api/articles	Feed with ?category, ?sub, ?sort, ?search, ?limit, ?cursor
GET	/api/articles/[id]	Single article with summary if available
POST	/api/summarize/[id]	Enqueue summarization; returns {jobId, status}
GET	/api/summarize/[id]/status	Poll summarization job status
GET	/api/source-health	Source health snapshots (admin-gated)

Table 12: Public API Endpoints

Internal Mutations (Server Actions, not Route Handlers)

```
features/admin/actions.ts -> createSource, updateSource, toggleSource
features/admin/actions.ts -> flagSummary, regenerateSummary
features/feed/actions.ts -> savePreference, setFavoriteCategory
```

10.2 Authentication and Authorization

Session auth via Auth.js v5 with HttpOnly session cookies. Authentication tokens are stored in HttpOnly cookies, never localStorage or sessionStorage. proxy.ts checks cookie presence for protected routes. Two roles: reader (default) and admin. Admin routes are protected at the proxy.ts level and the Server Action layer, providing defense in depth. Roadmap includes enterprise SSO via Auth.js v5 SAML/OIDC providers.

10.3 Error Handling

- JSON error format: {code: string, message: string, details?: unknown}.

- **UI:** non-blocking toast notifications for transient errors; inline empty states with recovery actions for feed errors.
- **All states handled:** loading (Suspense skeletons), error (error boundaries + error.tsx), empty (editorial empty states, not generic "no results"), success.

11. Caching, Performance and Scalability

11.1 Caching Strategy

Next.js 16 Cache Components provide the primary caching mechanism with explicit opt-in via the "use cache" directive. Unlike the implicit caching of previous versions, this model ensures that developers consciously choose what to cache and for how long, eliminating the silent caching bugs that plagued earlier releases. The cacheLife API provides named profiles (e.g., "seconds", "minutes", "hours", "days") that can be customized, and cacheTag enables targeted invalidation via revalidateTag.

11.2 Performance Targets

Metric	Target	Mechanism
Feed API p95	<= 500ms	Redis feed slices + optimized PG queries
Page FCP (feed)	<= 800ms	PPR prerendered shell via Cache Components
Page LCP (feed)	<= 1.5s	Streamed RSC with Suspense boundaries
Summarization trigger	Return <= 200ms	Enqueue only; async job processing
Search p95	<= 300ms	GIN-indexed tsvector with fastupdate=off

Table 13: Performance Targets

11.3 Scalability

- **Web App:** Stateless Next.js instances behind load balancer; scales horizontally. Server Components push computation to the server for better performance, smaller bundles, and fewer hydration issues.
- **Worker Service:** BullMQ supports multiple workers consuming the same queue with configurable concurrency. KEDA (Kubernetes Event-Driven Autoscaling) can scale workers based on queue depth. BullMQ can handle substantial load and is perfect for autoscaling with min/max replica counts.
- **Database:** PostgreSQL with read replicas for feed queries; writer for ingestion/summarization. Ingestion workers are I/O-bound (network fetches) and can run high concurrency; summarization workers are AI-API-bound and should rate-limit to avoid quota exhaustion.
- **Redis:** Managed Redis (Upstash or ElastiCache node-based clusters) with AOF persistence and noeviction policy. Upstash Redis works with BullMQ for persistent workers but is NOT suitable for serverless/Lambda worker patterns.

12. Observability and Operations

12.1 BullMQ Dashboard

The Worker Service exposes a BullMQ Board instance (or equivalent) for job monitoring. BullMQ provides a real-time dashboard to see exactly what is stuck, which is critical for a 200-source ingestion system with summarization pipelines. For production deployments requiring more than the open-source Bull Board offers, Taskforce.sh provides a professional dashboard built by the BullMQ team with real-time monitoring, alerts, metrics, and team collaboration features. Monitored queues: ingest, summarize, compute-importance, refresh-feed-slice.

12.2 Metrics

Category	Metrics
Ingestion	jobs/min, success rate, avg duration, new articles/run per source
Summarization	enqueue rate, completion rate, p95 latency, token usage/day, cost proxy
API	QPS per endpoint, p50/p95/p99 latency, error rates
Feed freshness	articles-per-category in last 24h (tracked via hourly snapshot job)

Table 14: Operational Metrics

12.3 Logging and Tracing

- Structured JSON logs from both Web App and Worker with correlation IDs on every log line.
- Request ID propagated through BullMQ job metadata for end-to-end tracing.
- Error boundaries capture client-side errors with source context.

12.4 Alerting

Condition	Alert Channel
Source ingestion failed 3x consecutive	PagerDuty / Slack
Summarization error rate > 5% over 1h	Slack warning
Feed freshness < 10 articles in top category	PagerDuty
API p95 > 1s for 5 minutes	PagerDuty
BullMQ DLQ > 50 items	Slack

Table 15: Alerting Rules

13. AI Governance

13.1 Disclosure Policy (Research-Grounded)

This section is substantially revised based on the AI trust research findings. The central paradox governing AI disclosure is that regulations mandate disclosure (EU AI Act Article 50 requires AI content disclosure + machine-readable markers by August 2026; California SB 942 effective October 2025), but research consistently shows that disclosure reduces trust even when content is identical. The "disclosure penalty" is real and measurable. However, the University of Florida (2026) found a "collaboration premium" where AI+human content is trusted more than either alone, and Pew Research found that while only 1% of users click citation links, their presence functions as a powerful trust signal.

Every AI summary MUST include, by default and without exception:

- **Provenance disclosure:** "AI-generated summary using [Model Name]" with machine-readable markers for regulatory compliance.
- **Source citations:** List of specific articles and sources the summary was derived from, with inline numbered superscripts linking to source articles (Perplexity hover-preview pattern).
- **Generation timestamp:** When this summary was generated.
- **Human verification status:** Whether it has been reviewed by an editor, creating the "collaboration premium" trust effect.
- **"Verify with original":** Persistent link to the source article, always a primary action.

13.2 Summary Quality Standards

- **Fidelity over brevity:** The goal is clarity without distortion. Shorter must still mean true. AI prompt design enforces factual-only output with no interpretation or opinion injection.
- Summaries do NOT speculate about causes or effects beyond what the source article states.
- Structured output is enforced via Zod schema validation: unstructured model output is rejected, not displayed.

13.3 Quality Control Process

- **Automated:** Flag summaries where model confidence is low OR content retrieval was partial.
- **Manual:** Admin samples 5% of summaries per category per week.
- **User reporting:** "Report inaccuracy" action on every summary panel, routing to `summary_status = "needs_review"`.
- **Regeneration:** Admin can trigger regeneration with updated prompt version.

13.4 Security

- AI API keys stored in environment secrets, never in database or logs.
- Input sanitization on article content before prompting (strip injection attempts).

- Output filtered through Zod schema: unstructured model output rejected.
- Rate limiting on /api/summarize/[id]: max 5 requests/min per IP (anonymous) or 20/min (authenticated).

14. Rollout Plan

Phase 1 - Production Foundation (MVP)

The first phase delivers the core platform on the new architecture. This includes the Next.js 16 web app with App Router, PPR, and Cache Components; the Worker service with BullMQ queues (ingest, summarize, compute-importance); PostgreSQL 17 schema with GIN FTS indexes; core feed, topic navigation, article detail, and source-cited AI summaries; admin source management and BullMQ job monitor; and Auth.js v5 for admin protection. The UI follows the "Editorial Dispatch" design direction with Newsreader + Space Grotesk typography, the Dispatch color token system, and three-column adaptive layout.

Phase 2 - Performance and Personalization

The second phase optimizes for scale and introduces personalization features. Redis feed slices for hot categories reduce database load. BM25 pg_textsearch for relevance-ranked search eliminates the ts_rank bottleneck. User preference persistence and favorite categories enable session continuity. The media bias indicators feature (inspired by Ground News) adds bias ratings and blindspot detection to source metadata. Enhanced admin summarization audit dashboard and feed freshness alerting runbooks round out operational maturity. A/B testing measures the citation presence trust uplift to validate the core AI disclosure strategy.

Phase 3 - Semantic Intelligence and Enterprise

The third phase introduces semantic search via pgvector with HNSW indexes and Reciprocal Rank Fusion (k=60) for hybrid keyword + semantic search, all within the same PostgreSQL database with no separate infrastructure. ML-based topic classification automates categorization for new sources. Enterprise SSO via Auth.js v5 SAML/OIDC providers enables organizational adoption. Multi-tenant source collections allow enterprise clients to define curated source sets. Advanced personalization uses behavior-based topic weighting. The "Opposite Sides" perspective view for polarized topics (inspired by Particle News) provides multi-perspective coverage analysis.

15. Risk Register

Risk	Likelihood	Impact	Mitigation
AI disclosure erodes user trust	High (research -validated)	High	Source-cited disclosure model; user testing; "collaboration premium" via editor badges

Next.js 16 Cache Components bugs in edge cases	Medium	High	Strong observability; cache-busting escape hatches; cacheTag for targeted invalidation
RSS source instability / format drift	High	Medium	Graceful parsing degradation; SourceHealthSnapshot alerts; adaptive polling intervals
AI summarization API cost overrun	Medium	Medium	Token budgets per job; summarization rate limiting; model tiering (Haiku for cost, Sonnet for quality)
PG FTS insufficient for scale (>10M articles)	Low (V1)	High	pg_textsearch BM25 as bridge; pgvector + external search on roadmap; partitioning strategy
BullMQ Redis single point of failure	Low	High	Managed Redis (Upstash/ElastiCache) with AOF persistence; noeviction policy; dedicated instance
EU AI Act compliance gaps	Medium	High	Machine-readable AI markers; disclosure by default; audit trail for every summary
Managed PG lacks BM25 extension	High	Medium	Self-hosted PG with pg_textsearch for search tier; native FTS as fallback on managed PG

Table 16: Risk Register

Appendix A: Technology Stack Summary

Layer	Technology	Version
Web Framework	Next.js	16
UI Runtime	React	19.2
Language	TypeScript	5.x (strict mode)
Styling	Tailwind CSS	v4
Component Library	Shadcn UI + Radix	Latest
ORM	Drizzle ORM	Latest
Validation	Zod	3.x
Auth	Auth.js	v5
Database	PostgreSQL	17
FTS	pg_textsearch (BM25) + tsvector GIN	1.x
Queue	BullMQ	5.x
Queue Backend	Redis (Upstash managed)	7.x
Worker Runtime	Node.js	24 LTS
AI Client	Anthropic SDK + OpenAI SDK	Latest

Containerization	Docker + Docker Compose	Latest
------------------	-------------------------	--------

Table 17: Technology Stack

Appendix B: Key Research Findings

Area	Draft	Refined	Evidence
Queue	SQS, RabbitMQ, or Redis-backed (TBD)	BullMQ on Redis (definitive)	14M+ npm downloads; job priorities, dashboard, TypeScript-native
Caching	revalidate segment caching (old model)	PPR + Cache Components (opt-in model)	Next.js 16 official release; explicit use cache directive
Network boundary	middleware.ts	proxy.ts	Next.js 16 release notes; codemod available
AI disclosure	Simple label ("AI-generated summary")	Source-cited, provenance-first disclosure system	Toff and Simon 2025; ShapeofAI patterns; Pew Research; UF 2026
FTS	PostgreSQL FTS (implied basic tsvector)	GIN tsvector + pg_textsearch BM25; pgvector roadmap	pg_textsearch v1.0 (March 2026); 138M docs benchmarked
SQLite	Used for local dev and testing	Removed; PostgreSQL container for all envs	Operational clarity; schema drift prevention
ORM	Not specified	Drizzle ORM (TypeScript-strict, lightweight)	Production consensus 2025-2026
Typography	Newsreader or equivalent	Newsreader + Space Grotesk + JetBrains Mono	Deliberate system; anti-generic mandate; 2026 typography trends
UI design direction	Editorial-industrial (general)	"Editorial Dispatch" with full token system	Anti-generic mandate; M3 Expressive research; editorial UX patterns
React features	React 19 (generic)	React 19.2 - View Transitions + Activity	Next.js 16 + React 19.2 changelog; specific feature use cases
Bias detection	Not present	Roadmap feature inspired by Ground News	Only platform with systematic bias ratings; market gap identified
Compliance	Not addressed	EU AI Act Art. 50 + CA SB 942 compliance	Regulatory research; Aug 2026 and Oct 2025 effective dates

Table 18: Draft vs. Refined Changes